Generative AI Part 3

Deepika Kanchi

October 2026

Contents

1 Introduction **3**
- 1.1 Applications of Transformers . . . 3
- 1.2 Why Transformers Were Introduced . . . 3
- 1.3 Overall Transformer Architecture . . . 4
- 1.4 Input Processing . . . 6
  - 1.4.1 Tokenization . . . 6
  - 1.4.2 Input Embedding . . . 6
  - 1.4.3 Positional Encoding . . . 6
- 1.5 Transformer Encoder . . . 7
- 1.6 Encoder Flow . . . 7
  - 1.6.1 Self-Attention . . . 8
  - 1.6.2 Query, Key, and Value . . . 8
  - 1.6.3 Meaning of Query, Key, and Value . . . 8
  - 1.6.4 Attention Score . . . 9
  - 1.6.5 Softmax . . . 9
  - 1.6.6 Weighted Sum of Values . . . 9
  - 1.6.7 Self-Attention Flow . . . 9
- 1.7 Multi-Head Attention . . . 10
- 1.8 Residual Connections and Layer Normalization . . . 10
  - 1.8.1 Residual Connection . . . 10
  - 1.8.2 Layer Normalization . . . 11
- 1.9 Position-Wise Feed-Forward Network . . . 11
- 1.10 Complete Encoder Block . . . 12
- 1.11 Transformer Decoder . . . 13
  - 1.11.1 Decoder Flow . . . 13
  - 1.11.2 Outputs Shifted Right . . . 13
  - 1.11.3 Masked Self-Attention . . . 14
  - 1.11.4 Encoder-Decoder Cross-Attention . . . 15
  - 1.11.5 Complete Decoder Block . . . 15
  - 1.11.6 Linear Layer and Softmax . . . 16
- 1.12 Linear Layer . . . 16
- 1.13 Softmax . . . 16

2 Pre-training vs Fine-tuning **17**

1 Introduction

A Transformer is a deep learning architecture designed to process sequential data using attention mechanisms. It was introduced in the 2017 research paper titled *Attention Is All You Need*. Transformers are widely used in Natural Language Processing, computer vision, speech processing, code generation, and multimodal Artificial Intelligence. Unlike Recurrent Neural Networks and Long Short-Term Memory networks, Transformers do not process tokens strictly one after another during training. Instead, they use self-attention to examine relationships between tokens and can process multiple tokens in parallel.

1.1 Applications of Transformers

Transformers are used in:

- Machine translation
- Text generation
- Question answering
- Text summarization
- Sentiment analysis
- Search engines
- Code generation
- Image generation
- Speech recognition
- Multimodal AI systems

Models such as GPT, BERT, T5, Codex, and several text-to-image systems use Transformer-based components.

1.2 Why Transformers Were Introduced

Before Transformers, sequential data was commonly processed using:

- Recurrent Neural Networks
- Long Short-Term Memory networks
- Gated Recurrent Units

These models process tokens sequentially. For example, consider the sentence:

> "The animal did not cross the road because it was tired."

To understand the word "it," the model must connect it with the word "animal," even though several words appear between them. Traditional recurrent models can struggle with such long-range dependencies. They are also difficult to parallelize because each step depends on the previous hidden state. Transformers solve these problems using self-attention.

Main Advantages

- Tokens can be processed in parallel during training.
- Long-range dependencies can be captured efficiently.
- Models can be scaled to large datasets and parameter counts.
- Attention provides a direct path between related tokens.
- The architecture can be adapted for many types of data.

1.3 Overall Transformer Architecture

The original Transformer has two major parts:

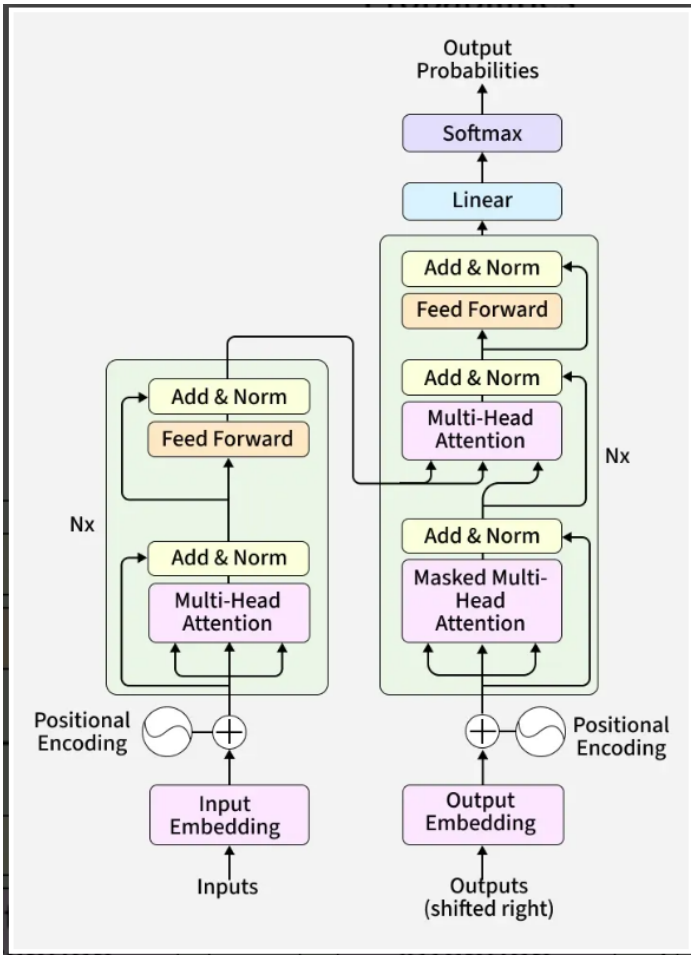
1. Encoder
2. Decoder

The complete flow is:

Input Sentence
↓
Tokenization
↓
Input Embedding and Positional Encoding
↓
Encoder Stack
↓
Contextual Input Representations
↓
Decoder Stack
↓
Linear Layer
↓
Softmax
↓
Output Token Probabilities

The encoder understands the input sequence. The decoder uses the encoder representation and previously generated tokens to produce the output sequence. For machine translation:

Input: "I love AI" Output: "J'aime l'IA"


Output Probabilities
Softmax
Linear
Add & Norm
Feed Forward
Add & Norm
Multi-Head Attention
Add & Norm
Masked Multi-Head Attention
Nx
Add & Norm
Feed Forward
Add & Norm
Multi-Head Attention
Nx
Positional Encoding
Positional Encoding
Input Embedding
Output Embedding
Inputs
Outputs (shifted right)

1.4 Input Processing

Before the input reaches the encoder, it passes through tokenization, embedding, and positional encoding.

1.4.1 Tokenization

Tokenization divides text into smaller units called tokens. For example:

`I love artificial intelligence`

may become:

`[I] [love] [artificial] [intelligence]`

A subword tokenizer may divide a word into smaller units:

`unbelievable` $\rightarrow$ `[un] [believ] [able]`

Each token is assigned a token ID from the model vocabulary.

1.4.2 Input Embedding

Computers cannot process words directly. Therefore, each token ID is converted into a numerical vector. For example:

$$\text{I} \rightarrow [0.2, -0.4, 0.8, \ldots]$$

$$\text{love} \rightarrow [0.7, 0.1, -0.3, \ldots]$$

$$\text{AI} \rightarrow [-0.6, 0.9, 0.2, \ldots]$$

If the model dimension is $d_{\text{model}} = 512$, each token is represented by a vector containing 512 values. The embedding layer learns meaningful relationships between tokens. Tokens with related meanings may obtain similar vector representations.

1.4.3 Positional Encoding

Self-attention processes input tokens simultaneously. Therefore, the Transformer requires additional information about token order. For example, these sentences contain similar words but different meanings:

"Dog bites person." "Person bites dog."

The positional encoding is added to the token embedding:

$$X_i = E_i + P_i$$

where:

- E_i is the embedding of token i

- P_i is the positional encoding of token i
- X_i is the final representation supplied to the model

The original Transformer uses sine and cosine functions:

$$PE(pos, 2i) = \sin\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right)$$

$$PE(pos, 2i+1) = \cos\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right)$$

The simple interpretation is:

- Token embedding represents what the token means.
- Positional encoding represents where the token occurs.

1.5 Transformer Encoder

The encoder creates contextual representations of the input sequence. An encoder block contains:

1. Multi-head self-attention
2. Residual connection and layer normalization
3. Position-wise feed-forward network
4. Residual connection and layer normalization

The encoder block is repeated N times. In the original Transformer, $N = 6$.

1.6 Encoder Flow

Input Embedding + Positional Encoding
↓
Multi-Head Self-Attention
↓
Add and Layer Normalization
↓
Feed-Forward Network
↓
Add and Layer Normalization
↓
Encoder Output

1.6.1 Self-Attention

Self-attention allows every token to examine every other token in the same input sequence. Consider:

> "The animal did not cross the road because it was tired."

When processing the token "it," self-attention can assign a high attention score to the token "animal." As a result, the representation of "it" contains contextual information about "animal."

1.6.2 Query, Key, and Value

Each input token is transformed into three vectors:

- Query vector Q
- Key vector K
- Value vector V

These vectors are calculated as:

$$Q = XW^Q$$
$$K = XW^K$$
$$V = XW^V$$

where:

- X is the input representation
- W^Q is the learned Query weight matrix
- W^K is the learned Key weight matrix
- W^V is the learned Value weight matrix

1.6.3 Meaning of Query, Key, and Value

The Query asks:

> What information is this token looking for?

The Key indicates:

> What type of information does this token contain?

The Value provides:

> What information should be passed to the requesting token?

A useful analogy is a library search:

- Query represents the requested topic.
- Key represents the label of each book.
- Value represents the information contained in the book.

1.6.4 Attention Score

The Query of one token is compared with the Keys of all tokens using a dot product:

$$QK^T$$

A high dot product represents a strong relationship. A low dot product represents a weak relationship. The score is scaled by the square root of the Key dimension:

$$\frac{QK^T}{\sqrt{d_k}}$$

The scaling prevents the values from becoming too large and improves the stability of the Softmax function.

1.6.5 Softmax

Softmax converts the attention scores into normalized weights:

$$A = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)$$

Example attention weights may be:

Token	Attention weight
The	0.05
animal	0.70
road	0.05
it	0.10
tired	0.10

The weights add up to 1.

1.6.6 Weighted Sum of Values

The attention weights are multiplied by the Value vectors:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

The output is a context-aware representation of every token.

1.6.7 Self-Attention Flow

Input Representations

↓

Create Queries, Keys, and Values

↓

Calculate QK^T

$$\downarrow$$
Scale by $\sqrt{d_k}$
$$\downarrow$$
Apply Softmax
$$\downarrow$$
Multiply by Values
$$\downarrow$$
Contextual Representations

1.7 Multi-Head Attention

A single attention operation may learn only one type of relationship. Multi-head attention performs several attention operations in parallel. Different attention heads may learn different relationships:

- Head 1 may learn subject-verb relationships.
- Head 2 may learn pronoun references.
- Head 3 may learn nearby word relationships.
- Head 4 may learn long-range dependencies.
- Head 5 may learn semantic relationships.

For attention head i:

$$head_i = \text{Attention}\left(QW_i^Q, KW_i^K, VW_i^V\right)$$

The outputs of all heads are concatenated:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(head_1, \ldots, head_h)W^O$$

where W^O is a learned output projection matrix. Therefore, multi-head attention allows the model to examine a sequence from several representational perspectives.

1.8 Residual Connections and Layer Normalization

Every major Transformer sublayer is followed by a residual connection and layer normalization.

1.8.1 Residual Connection

A residual connection adds the original input to the processed output:

$$Y = X + \text{SubLayer}(X)$$

For an attention sublayer:

$$Y = X + \text{MultiHeadAttention}(X)$$

Residual connections:

- Preserve the original information
- Improve gradient flow
- Make deep models easier to train
- Reduce information loss across layers

1.8.2 Layer Normalization

Layer normalization normalizes the activations of each token representation. A simplified expression is:

$$\text{LayerNorm}(x) = \gamma \frac{x - \mu}{\sqrt{\sigma^2 + \epsilon}} + \beta$$

where:

- μ is the mean
- σ^2 is the variance
- ϵ avoids division by zero
- γ and β are learned parameters

In the original post-normalization Transformer:

$$\text{Output} = \text{LayerNorm}\left(X + \text{SubLayer}(X)\right)$$

Some modern Transformers use pre-normalization, where layer normalization is applied before the attention or feed-forward operation.

1.9 Position-Wise Feed-Forward Network

After self-attention, every token representation passes through a feed-forward neural network. The original formulation is:

$$\text{FFN}(x) = \text{ReLU}(xW_1 + b_1)W_2 + b_2$$

The network contains:

1. First linear transformation
2. Nonlinear activation
3. Second linear transformation

For example:

512 dimensions
↓
2048 dimensions
↓
ReLU activation
↓
512 dimensions

The same network parameters are applied to every token position, but each position is processed independently. The difference between attention and the feed-forward network is:

- Attention collects information from other tokens.
- The feed-forward network processes the collected information.

Modern Transformer models may use activations such as GELU or SwiGLU instead of ReLU.

1.10 Complete Encoder Block

Let the encoder input be X. First, multi-head self-attention is applied:

$$A = \text{MultiHeadAttention}(X, X, X)$$

Then, the first Add and Norm operation is applied:

$$Y = \text{LayerNorm}(X + A)$$

Next, the feed-forward network is applied:

$$F = \text{FFN}(Y)$$

Finally, the second Add and Norm operation is applied:

$$Z = \text{LayerNorm}(Y + F)$$

The representation Z becomes the output of one encoder layer. If there are N encoder layers, the output from one layer becomes the input to the next layer. The final encoder output can be written as:

$$H = [h_1, h_2, \ldots, h_n]$$

Each vector h_i represents one input token within the context of the complete input sequence.

1.11 Transformer Decoder

The decoder generates the output sequence one token at a time. A decoder block contains:

1. Masked multi-head self-attention
2. Add and Layer Normalization
3. Encoder-decoder cross-attention
4. Add and Layer Normalization
5. Feed-forward network
6. Add and Layer Normalization

The block is repeated N times.

1.11.1 Decoder Flow

Previous Output Tokens Shifted Right
↓
Output Embedding and Positional Encoding
↓
Masked Multi-Head Self-Attention
↓
Add and Layer Normalization
↓
Encoder-Decoder Cross-Attention
↓
Add and Layer Normalization
↓
Feed-Forward Network
↓
Add and Layer Normalization
↓
Linear Layer
↓
Softmax
↓
Next Output Token

1.11.2 Outputs Shifted Right

During training, the target sequence is shifted one position to the right before it is supplied to the decoder. Suppose the target is:

```
I love AI
```

The decoder input is:

<START> I love AI

The expected output is:

I love AI <END>

The token-level relationship is:

Decoder input	Expected next token
<START>	I
I	love
love	AI
AI	<END>

This teaches the decoder to predict the next token from the previous tokens.

1.11.3 Masked Self-Attention

The first attention layer in the decoder is masked self-attention. The decoder must not see a future token before predicting that token. For example, when predicting the final word in:

I love ___

the decoder can use:

I love

but it must not see the correct future token:

AI

A causal mask is added to the attention scores:

$$\text{MaskedAttention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}} + M\right)V$$

A three-token causal mask can be written as:

$$M = \begin{bmatrix} 0 & -\infty & -\infty \\ 0 & 0 & -\infty \\ 0 & 0 & 0 \end{bmatrix}$$

After Softmax, blocked future positions receive probability zero. The visibility pattern is:

Current token	Token 1	Token 2	Token 3
Token 1	Visible	Hidden	Hidden
Token 2	Visible	Visible	Hidden
Token 3	Visible	Visible	Visible

This is also called:

- Causal attention
- Autoregressive attention
- Look-ahead masking

1.11.4 Encoder-Decoder Cross-Attention

The second attention layer in the decoder is called cross-attention. Cross-attention connects the decoder to the final encoder output. In cross-attention:

$$Q = \text{decoder representation}$$

$$K = \text{encoder representation}$$

$$V = \text{encoder representation}$$

The calculation is:

$$\text{CrossAttention}(Q_d, K_e, V_e) = \text{softmax}\left(\frac{Q_d K_e^T}{\sqrt{d_k}}\right) V_e$$

where:

- Q_d comes from the decoder
- K_e comes from the encoder
- V_e comes from the encoder

For translation, when the decoder generates the translation of the word "love," cross-attention can focus on the encoder representation of "love." The two decoder attention mechanisms have different purposes:

- Masked self-attention examines previously generated output tokens.
- Cross-attention examines the original input sequence.

1.11.5 Complete Decoder Block

Let the decoder input be D, and let the encoder output be H. First, masked self-attention is calculated:

$$A_1 = \text{MaskedMultiHeadAttention}(D, D, D)$$

Then Add and Norm is applied:

$$Y_1 = \text{LayerNorm}(D + A_1)$$

Cross-attention is then calculated:

$$A_2 = \text{MultiHeadAttention}(Y_1, H, H)$$

Here, Y_1 supplies the Queries, while H supplies the Keys and Values. The second Add and Norm operation is:

$$Y_2 = \text{LayerNorm}(Y_1 + A_2)$$

The feed-forward network is applied:

$$F = \text{FFN}(Y_2)$$

The final Add and Norm operation is:

$$O = \text{LayerNorm}(Y_2 + F)$$

The output O becomes the input to the next decoder layer or the final Linear layer.

1.11.6 Linear Layer and Softmax

The final decoder representation must be converted into scores for all tokens in the vocabulary.

1.12 Linear Layer

Suppose:

- Model dimension is 512
- Vocabulary size is 50,000

The Linear layer maps the 512-dimensional decoder output into 50,000 scores:

$$\text{logits} = OW_{\text{vocab}} + b$$

Each score corresponds to one vocabulary token. Example:

Token	Logit
AI	8.2
ML	6.1
coding	4.7
apple	-1.3

These scores are called logits.

1.13 Softmax

Softmax converts logits into probabilities:

$$P(y_t = j) = \frac{e^{z_j}}{\sum_{k=1}^{|V|} e^{z_k}}$$

Example:

Token	Probability
AI	0.72
ML	0.18
coding	0.07
apple	0.03

The probabilities add up to 1. The model may select the next token using:

- Greedy decoding
- Beam search
- Temperature sampling
- Top-k sampling
- Top-p or nucleus sampling

2 Pre-training vs Fine-tuning

- GPT → Pre-trained on Internet-scale text → Fine-tuned for chat and instructions
- BERT → Pre-trained with MLM → Fine-tuned for classification and NLP tasks

Pre-training is the process of training a model on massive datasets to learn general language, visual, or coding knowledge. Fine-tuning is the process of further training the pre-trained model on a smaller task-specific dataset so that it performs better on a particular application such as sentiment analysis, medical diagnosis, customer support, or code generation.